

Documentação Técnica de Replicação

Adaptação do Portfólio Cyd Stumpel para a stack Next.js + Supabase + GSAP

Documento de referência para implementação por um desenvolvedor fullstack/mobile com experiência em TypeScript, Next.js, Supabase, Prisma e Tailwind. Esta versão substitui o stack WordPress/PHP original por uma arquitetura headless 100% Next.js, mantendo o sistema de animações (GSAP + ScrollTrigger + Lenis + View Transitions API) que é o grande diferencial visual do site original.

Sumário

1. Visão Geral do Projeto

1.1 Resumo da proposta visual

Design limpo e moderno, com forte ênfase em tipografia editorial (fonte serifada para títulos + sans-serif para corpo) e animações fluidas. A experiência é guiada por transições de página suaves e animações controladas por scroll que revelam conteúdo de forma dinâmica. A paleta de cores é configurável via variáveis CSS, permitindo temas por página/seção.

1.2 Objetivos de UX

Navegação intuitiva e visualmente rica, onde o conteúdo é o protagonista. As animações guiam o usuário e adicionam personalidade sem comprometer acessibilidade nem performance — o site deve demonstrar domínio técnico em desenvolvimento criativo e design responsivo.

1.3 Público-alvo

Clientes potenciais, recrutadores e a comunidade de desenvolvimento web interessada em animação front-end e design de interfaces.

1.4 Principais diferenciais da experiência

- Transições de página nativas via CSS View Transitions API combinadas com Swup (ou roteamento próprio do Next.js + View Transitions API).
- Animações controladas por scroll usando GSAP ScrollTrigger e, onde possível, CSS Scroll-Driven Animations nativas.
- Tipografia dinâmica com hierarquia bem definida via classes utilitárias do Tailwind.
- Variáveis CSS para temas e cores configuráveis por página/seção (data-accent-color).
- Scroll suave com inércia via Lenis.

2. Arquitetura e Stack Tecnológica (Adaptada)

A stack original (WordPress + PHP + tema customizado) foi substituída por uma arquitetura totalmente baseada em Next.js, alinhada ao seu fluxo de trabalho atual com TypeScript, Supabase e Prisma.

Tecnologia	Função	Substitui / Equivalência ao original	Observações
Next.js 16+ (App Router)	Framework principal — rotas, SSR/SSG, layouts	WordPress + PHP (front-end e roteamento)	Usar Server Components para conteúdo estático/SEO e Client Components para animações
TypeScript	Tipagem em todo o projeto	PHP / JS Vanilla	Aproveita sua stack já consolidada
Supabase (Postgres + Storage)	Headless CMS — projetos, posts, configs de tema	WordPress (CMS + banco de dados)	Tabelas: projects, posts, site_settings, theme_config
Prisma	ORM de acesso ao Supabase/Postgres	—	Camada de tipagem sobre as tabelas do Supabase
Tailwind CSS + CSS Variables	Design system, utilitários e temas dinâmicos	CSS customizado do tema WordPress	Variáveis CSS para --color-accent, --highlight-color etc., aplicadas via Tailwind config + data-attributes
GSAP + ScrollTrigger	Animações complexas e scroll-driven	GSAP (mantido)	Mesma lib do original — controle fino de timelines
@studio-freight/react-lenis	Scroll suave com inércia	Lenis (mantido)	Wrapper React oficial para Lenis
CSS View Transitions API	Transições entre rotas do Next.js	Swup + View Transitions API	Next.js 13+ tem suporte experimental;

			fallback com GSAP se necessário
CSS Scroll-Driven Animations	Paralaxe e revelação via scroll nativo	CSS Scroll-Driven Animations (mantido)	Usar como complemento ao ScrollTrigger em navegadores compatíveis

Observações de adaptação

- Three.js não é necessário na página principal — pode ser usado pontualmente em cards de projetos que demonstrem trabalhos 3D, como módulo carregado sob demanda (dynamic import).
- SEO: substituir Yoast SEO pelo módulo de Metadata API nativo do Next.js (generateMetadata por rota).
- Cache/performance: usar ISR (Incremental Static Regeneration) do Next.js para páginas de projeto/blog vindas do Supabase.

3. Design System Completo

O design system é implementado via Tailwind config + variáveis CSS globais, permitindo override por página através de data-attributes (ex.: data-accent-color).

3.1 Tipografia

- **Fonte de títulos (serifada):** equivalente a Instrument Serif — sugestão:
- Carregar via next/font/google para otimização automática (font-display, subsetting).
- **Fonte de corpo (sans-serif):** equivalente a Geist — usar Geist (já mantida pela Vercel, integração nativa com next/font)

Hierarquia tipográfica (classes utilitárias)

Classe Tailwind sugerida	Equivalente original	Uso
.text-huge-hero	huge-hero	Título principal do Hero, maior elemento da página
.text-large-heading	large-heading	Títulos de seção
.text-small-heading	small-heading	Subtítulos
.text-large-body	large-body	Texto de destaque/intro
.text-small-body	small-body	Texto corrido, menor tamanho
.uppercase tracking-wide	caps	Texto em maiúsculas (labels, tags)
.font-bold	font-bold	Peso forte em Geist

3.2 Cores (variáveis CSS + Tailwind)

Definir como CSS custom properties no globals.css e referenciar no tailwind.config via theme.extend.colors, permitindo override dinâmico por página.

```
:root {
  --color-background: #FFF5EE; /* seashell */
  --color-text: #0A0A0A;
  --color-accent: #DF4F2A; /* vermelho/laranja */
  --highlight-color: #E8FCC2; /* verde/amarelo vibrante */
}

/* Override por página via data-attribute */
[data-accent-color='blue'] { --color-accent: #2A6BDF; }
[data-accent-color='green'] { --color-accent: #2ADF7A; }
// tailwind.config.ts
colors: {
  background: 'var(--color-background)',
  text: 'var(--color-text)',
  accent: 'var(--color-accent)',
  highlight: 'var(--highlight-color)',
}
```

3.3 Grid e Layout

- Sistema de grid: usar Tailwind grid grid-cols-12 (desktop) com gap configurável via variável CSS --grid-gap.
- Breakpoints sugeridos: mobile (grid-cols-1 / 2), tablet (grid-cols-6), desktop (grid-cols-12).
- Largura do conteúdo principal: usar uma classe utilitária .content-container com max-width controlado por variável CSS, equivalente a --block-content-columns.

3.4 Componentes Principais

Componente	Descrição	Implementação sugerida
Header	Fixo no topo, logo + nav (About, Archive, Speaking, Blogs, Contact)	Server Component estático + position: sticky, link de email com mailto:
Hero Section	Nome/marca com texto duplicado e animado, fundo animado	Client Component — múltiplas cópias do <h1> posicionadas via CSS transform + GSAP timeline
Cards de Projeto	Imagem, título, ano, tags (Dev, Motion, GSAP, Three.JS), hover com border-radius animado	Componente reutilizável <ProjectCard />, dados vindos do Supabase (tabela projects)
Links com tooltip	Sublinhado animado + tooltip com imagem/vídeo no hover	Componente <LinkHover /> com Framer/GSAP para o tooltip; conteúdo via props
Footer	Nav duplicada + redes sociais (Github, Codepen, Bluesky, Mastodon, Instagram, LinkedIn, RSS)	Server Component estático, lista de links configurável via Supabase (tabela site_settings)

4. Mapeamento Completo das Animações

4.1 Page Load

- Componente <Loader /> renderizado no layout raiz.
- Estado isLoading controlado via useState + useEffect (window load / Lenis ready).
- Transição: opacity de 1 → 0 e pointer-events: none ao carregar, via classe condicional ou GSAP .to().

4.2 Scroll Animations

- Biblioteca: GSAP + ScrollTrigger, complementado por CSS Scroll-Driven Animations onde suportado.
- Título principal: efeito de paralaxe e rotação via ScrollTrigger com scrub: true.
- Cards de projeto: animação de entrada com rotação leve e delay incremental baseado no índice (stagger no GSAP, equivalente ao sibling-index() do CSS).
- Mudança de background ao longo do scroll: usar ScrollTrigger.create() alterando uma variável CSS --bg-progress consumida pelo background.

4.3 Hover Effects

- Links: tooltip (<LinkHover__Tooltip />) com imagem/vídeo, exibido via GSAP fade/scale no mouseenter.
- Cards de projeto: transição de border-radius e opacidade via Tailwind transition-all + variáveis --border-radius / --hover-radius.

4.4 Text Animations

- Título do Hero duplicado N vezes (mapeado via array em React), cada cópia com transform: rotate() translate() controlado por variáveis CSS --custom-rotate-N.
- Pequenos offsets aplicados via inline style ou classes utilitárias geradas dinamicamente.

4.5 Transições de Página

Substituindo Swup pelo roteamento nativo do Next.js App Router combinado com a View Transitions API:

```
// app/template.tsx ou hook customizado
import { useRouter } from 'next/navigation'
```

```
function navigateWithTransition(path: string) {
  if (!document.startViewTransition) {
    router.push(path)
    return
  }
  document.startViewTransition(() => {
    router.push(path)
  })
}
```

- Animações de scroll (ScrollTrigger) devem ser pausadas e refreshadas no evento de transição (ScrollTrigger.refresh() após a navegação).

4.6 Microinterações

- Botão 'Copiar email': `navigator.clipboard.writeText()` + feedback visual (texto muda para 'Copiado!' por 2s, com transição GSAP ou CSS).

5. Efeitos Visuais

- **Paralaxe:** ScrollTrigger com scrub em elementos de texto/imagem, velocidades diferentes por camada (data-speed).
- **Glassmorphism/Blur:** opcional — backdrop-blur do Tailwind em modais ou overlays.
- **Glow/Shadows:** box-shadow e drop-shadow utilitários do Tailwind, sutis.
- **Gradients:** bg-gradient-to-* do Tailwind para fundos e detalhes.
- **Scroll Smoothing:** @studio-freight/react-lenis envolvendo o layout raiz.

6. Estrutura das Seções

1. Hero Section — nome/marca com animação de texto duplicado
2. About Section — informações pessoais, habilidades, experiência
3. Work/Projects Section — galeria de projetos com cards interativos (dados do Supabase)
4. Speaking/Conteúdo Section — palestras, vídeos, lives (opcional)
5. Blog/Posts Section — lista de artigos recentes (dados do Supabase)
6. Contact Section — formulário de contato + informações
7. Footer — navegação duplicada + redes sociais

7. Responsividade

Breakpoint	Grid	Animações
Desktop (lg+)	12 colunas, espaçamento amplo	Todas as animações ativas (ScrollTrigger completo, paralaxe, tooltips)
Tablet (md)	6 colunas	Animações simplificadas — reduzir paralaxe e duração de transições
Mobile (sm)	1 coluna, elementos empilhados	Desativar tooltips com vídeo; reduzir/desativar paralaxe; manter transições de página leves

Sugestão: usar `matchMedia` do GSAP (`gsap.matchMedia()`) para registrar conjuntos de animações distintos por breakpoint, desligando efeitos pesados em mobile.

8. Performance

- **Otimização de imagens:** componente `next/image` com lazy loading automático e formatos modernos (AVIF/WebP).
- **Lazy Loading de componentes:** `dynamic()` do Next.js para módulos pesados (Three.js, GSAP plugins específicos).
- **Code Splitting:** automático pelo App Router (por rota); evitar imports globais de GSAP plugins não usados.
- **Otimização de animações:** GSAP + CSS Scroll-Driven Animations + View Transitions API aproveitando otimizações nativas do navegador.
- **Cache:** ISR/revalidate do Next.js para conteúdo do Supabase + cache de assets via CDN (Vercel/Cloudflare).

9. Guia de Implementação

9.1 Estrutura de Pastas

```
/src
/app          # Rotas do Next.js (App Router)
/(site)
  /page.tsx    # Home
  /about/page.tsx
  /work/page.tsx
  /work/[slug]/page.tsx
  /blog/page.tsx
  /blog/[slug]/page.tsx
  /contact/page.tsx
  /layout.tsx
  /template.tsx # View Transitions wrapper
  /globals.css
  /components
  /ui          # Botões, Cards, Header, Footer, LinkHover
  /sections    # Hero, About, Projects, Contact
  /hooks       # useLenis, useScrollTrigger, useViewTransition
  /lib
  /supabase.ts # client Supabase
  /prisma.ts   # client Prisma
  /animations  # timelines GSAP reutilizáveis
/prisma
/schema.prisma
```

9.2 Modelagem Supabase / Prisma (sugestão inicial)

```
model Project {
  id      String @id @default(uuid())
  title   String
  slug    String @unique
  year    Int
  tags    String[] // ex: ["Dev", "Motion", "GSAP", "Three.js"]
  coverImage String
  content String? // markdown/rich text
  accentColor String? // override do tema por projeto
  createdAt DateTime @default(now())
}
```

```
model Post {
  id      String @id @default(uuid())
  title   String
  slug    String @unique
  excerpt String?
  content String
  published Boolean @default(false)
  createdAt DateTime @default(now())
}
```

```
model SiteSettings {
  id      Int   @id @default(1)
  email   String
  socialLinks  Json // { github, codepen, bluesky, mastodon, instagram, linkedin, rss }
  defaultAccent String @default("#DF4F2A")
}
```

9.3 Ordem Ideal de Desenvolvimento

Fase 1 — Setup e Estrutura Básica (Dificuldade: Baixa)

- Inicializar projeto Next.js (App Router) + TypeScript + Tailwind.
- Configurar next/font para Geist e a fonte serifada de títulos.
- Configurar Lenis (@studio-freight/react-lenis) no layout raiz.
- Criar Header, Footer e seções com conteúdo estático/mock.

Fase 2 — Integração com Supabase (Dificuldade: Média)

- Configurar projeto Supabase + Prisma com o schema acima.
- Criar funções de fetch (server-side) para projects, posts e site_settings.
- Popular Hero, Work e Blog com dados reais via Server Components.

Fase 3 — Animações de Texto e Scroll (Dificuldade: Média-Alta)

- Implementar texto duplicado do Hero com GSAP timeline.
- Configurar ScrollTrigger para entrada/saída de elementos e paralaxe.
- Experimentar CSS Scroll-Driven Animations como complemento em navegadores compatíveis.
- Registrar gsap.matchMedia() para diferenciar comportamento por breakpoint.

Fase 4 — Transições de Página (Dificuldade: Alta)

- Implementar transições via View Transitions API integrada ao App Router.
- Fallback com GSAP para navegadores sem suporte (feature detection de document.startViewTransition).
- Garantir ScrollTrigger.refresh() após cada transição de rota.

Fase 5 — Polimento e Otimização (Dificuldade: Média)

- Refinar hover effects (LinkHover, cards) e microinteração do botão de copiar email.
- Otimizar imagens com next/image e lazy loading de componentes pesados.
- Testar responsividade e ajustar/desligar animações pesadas em mobile.
- Configurar Metadata API para SEO (substituindo Yoast).

10. Conclusão e Blueprint

Esta adaptação mantém o conceito central do portfólio original — navegação fluida com transições nativas, animações de scroll de alta qualidade e tipografia editorial — substituindo toda a camada WordPress/PHP por uma stack 100% TypeScript (Next.js + Supabase + Prisma), alinhada ao seu fluxo de trabalho atual e sem dependência de CMS tradicional.

Nível de Complexidade

Médio a Avançado. Requer domínio de animações com GSAP/ScrollTrigger, integração de View Transitions API com o App Router do Next.js, e modelagem básica de dados no Supabase.

Tempo Estimado

Para um desenvolvedor com sua experiência (Next.js + Supabase já no dia a dia): aproximadamente 2 a 4 semanas de trabalho focado, considerando as 5 fases acima, sendo a Fase 4 (transições de página) a de maior risco/tempo.

O Segredo do Sucesso

Combinar View Transitions API (nativa do navegador, leve) com GSAP ScrollTrigger e Lenis para criar uma experiência de navegação fluida e performática — exatamente como no original, porém sobre uma base 100% Next.js/Supabase que você já domina, sem o overhead de WordPress.